CYBERSECURITY AVANZATA · LEZIONE 5 DI 8

# OAuth, JWT e passkey

"Accedi con Google", i token che viaggiano nelle API REST e le chiavi che hanno preso il posto delle password: l'autenticazione come la costruiscono oggi le applicazioni vere.

5G TPSI

MFA, OAuth 2.0 e PKCE, OpenID Connect, JWT, WebAuthn e passkey

PER COMINCIARE

# Come accedi di solito a un'app nuova?

SONDAGGIO

- **A** Creo un account con e-mail e password
- **B** Accedi con Google, Apple o Microsoft
- **C** Con il numero di telefono e un codice SMS
- **D** Con una passkey (impronta o volto)
- **E** Non mi ricordo: faccio sempre "password dimenticata"

PIÙ FATTORI

# Non tutta la MFA vale uguale

| Secondo fattore | Come funziona | Punto debole |
|---|---|---|
| **SMS** | Codice inviato al numero di telefono | SIM swap: l'attaccante si fa intestare il tuo numero |
| **App con codice (TOTP)** | Codice da 6 cifre che cambia ogni 30 secondi | Un sito falso può chiederlo e usarlo subito |
| **Notifica push** | "Sei tu?" sul telefono, tocchi Sì | MFA fatigue: notifiche a raffica finché tocchi Sì |
| **Chiave FIDO / passkey** | Firma legata al dominio del sito | Resiste al phishing: sul sito falso non funziona |

QUIZ A TRABOCCHETTO

# Hai attivato i codici della app di autenticazione. Un sito clone della tua banca ti chiede password e codice, e tu li inserisci. Che cosa succede?

QUIZ

A Niente: il codice è valido solo sul sito vero

B L'attaccante li inoltra subito alla banca vera ed entra

C La banca blocca l'accesso perché il codice arriva da un altro IP

D Il codice scade prima che l'attaccante lo possa usare

DELEGARE SENZA DARE LA PASSWORD

# OAuth 2.0: le chiavi della macchina, non le chiavi di casa

Un'app di fotoritocco vuole leggere le tue foto su Drive. Con OAuth non le dai la password di Google: Google le dà un **token** limitato.

- Solo i permessi chiesti (**scope**): "leggi le foto", non "cancella la posta"
- Per un tempo limitato, e revocabile quando vuoi
- OAuth autorizza. **OpenID Connect**, sopra OAuth, aggiunge chi sei: è "Accedi con Google"

## I quattro attori

- **Utente**: tu, proprietario dei dati
- **Client**: l'app di fotoritocco
- **Authorization server**: la pagina di login di Google
- **Resource server**: l'API di Drive

PUZZLE

# Metti in ordine il flusso Authorization Code con PKCE

1. Il browser torna all'app con un codice monouso
2. L'app genera un segreto casuale e ne manda l'hash con la richiesta
3. L'app chiama l'API con Authorization: Bearer e il token
4. L'utente fa login e approva gli scope richiesti
5. L'app scambia codice e segreto originale con un access token
6. Il browser va alla pagina di login dell'authorization server

IL FORMATO DEI TOKEN

# JWT: tre parti separate da un punto

### Header

{"alg":"HS256","typ":"JWT"}

Algoritmo della firma

### Payload

{"sub":"studente42",
"ruolo":"studente",
"exp":1893456000}

I dati (claim)

### Firma

HMAC-SHA256(header.payload, segreto)

Chi non ha il segreto non può rifarla

> Header e payload sono solo codificati in **Base64url**, non cifrati: **chiunque li legge**. La firma garantisce che non sono stati modificati, non che sono segreti.

AL COMPUTER · 20 minuti

# Leggi un JWT senza conoscere il segreto

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJzdHVkZW50ZTQyIiwibm9tZSI6IlN0dWRlbnRlIGRpIHByb3ZhIiwicnVvbG8iOiJzdHVkZW50ZSIsImV4cCI6MTg5MzQ1NjAwMH0.ssNrSrqhoOiimbtQ0muTviGAP_K_cOHWiytyL1wRhT0

1. Copia il token in **Jwt.java** (il docente lo pubblica anche su Classroom, per non ricopiarlo a mano).
2. Dividilo con `split("\\.")` e decodifica le prime due parti con `Base64.getUrlDecoder()`.
3. Stampa header e payload e converti `exp` in data con `Instant.ofEpochSecond`.

**Controllo:** il payload contiene `"ruolo":"studente"` e `exp` corrisponde al **1° gennaio 2030**, 00:00 UTC.

AL COMPUTER · 25 minuti

# Promuoviti a docente (e fallisci)

```
Mac mac = Mac.getInstance("HmacSHA256");
mac.init(new SecretKeySpec(segreto, "HmacSHA256"));
byte[] firma = mac.doFinal((p[0] + "." + p[1]).getBytes(UTF_8));
String attesa = Base64.getUrlEncoder().withoutPadding()
    .encodeToString(firma);
```

1. Il docente ti dà il segreto. Scrivi `verifica(token)`: la firma ricalcolata deve essere uguale alla terza parte.
2. Ora cambia `"studente"` in `"docente"` nel payload, ricodificalo e rimonta il token con la **vecchia** firma.

**Controllo:** il token originale passa (la firma inizia con **ssNrSrqh**), quello modificato **no**. Per falsificarlo servirebbe il segreto.

PUZZLE

# Abbina ogni parola al suo ruolo

| | | |
|---|---|---|
| **Scope** | --- | Protegge il codice monouso da chi lo intercetta |
| **Access token** | --- | I permessi che il client chiede |
| **ID token** | --- | La chiave che il client presenta all'API |
| **PKCE** | --- | Il claim con la scadenza del token |
| **exp** | --- | Il JWT di OpenID Connect con l'identità dell'utente |

IL 1993

# Il problema non è nuovo

> **Su Internet nessuno sa che sei un cane.**
>
> **Peter Steiner**, didascalia di una vignetta, *The New Yorker*, 5 luglio 1993

Trent'anni dopo cerchiamo ancora il modo migliore per dimostrare chi siamo a un server lontano, senza consegnargli un segreto che si può rubare.

SENZA PASSWORD

# Passkey: una coppia di chiavi per ogni sito

- Alla registrazione il dispositivo crea una **coppia di chiavi** per quel sito: la pubblica va al server, la privata resta sul dispositivo
- Al login il server manda una sfida casuale, il dispositivo la **firma** dopo l'impronta, il volto o il PIN
- La firma vale solo per il **dominio** registrato: su un sito clone non parte
- Il server conserva solo chiavi pubbliche: un furto del database non rivela segreti

## Che cosa sono

Credenziali **FIDO2 / WebAuthn** (standard W3C e FIDO Alliance), sincronizzabili tra i dispositivi tramite il gestore delle password di Google, Apple, Microsoft o di terzi.

AL COMPUTER · 15 minuti

# Crea una passkey di prova e guarda che cosa riceve il server

1. Apri **webauthn.io**, il sito dimostrativo di WebAuthn. Inventa un nome utente di prova (non il tuo nome).
2. Premi **Register** e segui il browser (PIN di Windows Hello, telefono o chiave USB).
3. Premi **Authenticate**: ti serve una password? Apri il pannello dei dettagli e annota in **auth-cognome.md** che cosa riceve il server.
4. Alla fine elimina la passkey di prova dal gestore delle password del dispositivo.

**Controllo:** il login avviene senza password. Tra i dati inviati c'è una **chiave pubblica** alla registrazione e una **firma** all'accesso, mai l'impronta o il PIN.

QUIZ

# Un attaccante ruba il database di un sito che usa solo passkey. Che cosa ottiene?

QUIZ

A Chiavi pubbliche, inutili per entrare

B Le impronte digitali degli utenti

C Gli hash delle chiavi private, da attaccare offline

D I PIN dei telefoni degli utenti

OGGI

# Token anche per gli agenti AI





SFIDA A SQUADRE

# Apri il lucchetto



- Il codice è l'anno in cui scade il token del laboratorio.
- Per trovarlo non serve il segreto: basta leggerlo.

`codice`

BIGLIETTO D'USCITA

# Completa prima di uscire

Le notifiche push a raffica finché tocchi Sì si chiamano MFA _____.

OAuth autorizza; per sapere chi è l'utente serve OpenID _____.

Le parti di un JWT sono codificate in _____, non cifrate.

Il claim con la scadenza di un JWT si chiama _____.

Una passkey resiste al phishing perché la firma è legata al _____.

⏱ 3:00

RIEPILOGO

# Cosa portiamo a casa

- MFA: meglio di niente, ma SMS e codici si rubano
- OAuth: token limitati invece della password
- OpenID Connect: l'identità, con l'ID token
- Authorization Code con PKCE per tutti
- JWT: leggibile da tutti, firmato, con scadenza
- Il server fissa l'algoritmo e controlla exp, iss, aud
- Passkey: chiavi per sito, niente da rubare sul server
- Prossima lezione: anatomia di un'e-mail di phishing

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.